

Up next in Hands-on Labs →

Lab 03: Multibranch Foundations

 Hands-on lab · 50 min · Env: See book overview · Overview & prereqs · Download PDF · [Reset checks](#)

CI/CD ENGINEERING

Container Artifacts & Delivery: From
Source to Registry

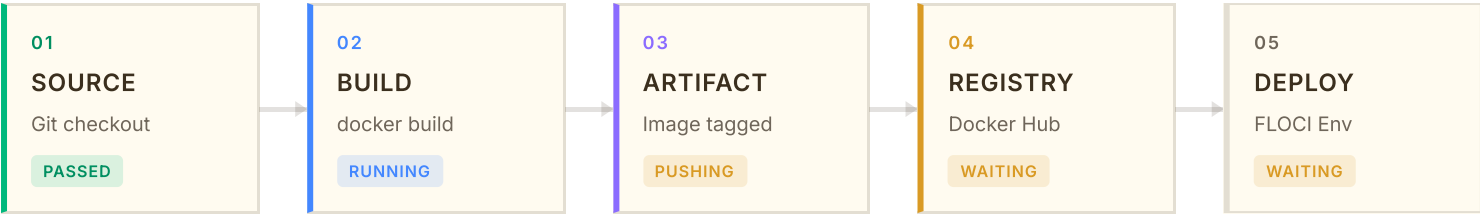
CONTINUOUS INTEGRATION
CONTAINER DELIVERY
SIMULATED DEPLOYMENT

Nabawy



01 / The Container Lifecycle

Moving from source code to a running application requires a deterministic path. The code must be compiled, packaged with its dependencies, and shipped as an immutable artifact. In modern CI/CD, this artifact is a Docker image.



WHAT

A pipeline that pulls source code, builds a Docker image, tags it with a unique build identifier, pushes it to a central registry, and optionally deploys it to a target environment.

WHY

Containerizing the application ensures environment parity. The exact artifact that passed CI is the one deployed, eliminating "it works on my machine" discrepancies.

WHEN

On every commit or merge to the main branch. The pipeline triggers automatically via webhook, executing the build and push sequence.

WHAT CHANGES

The release unit shifts from raw source code to a versioned, immutable container image. The registry becomes the source of truth for deployable software.

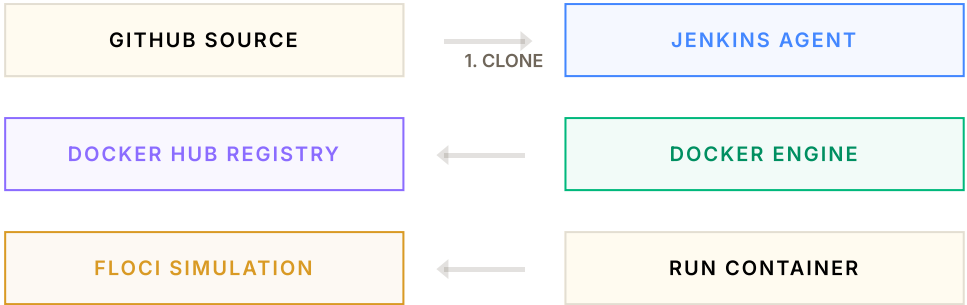
WHY THIS EXISTS

Without a standardized packaging format, deployment scripts must understand the application's dependency tree. A Docker image bundles the OS, libraries, and code into a single, static unit, radically simplifying the deployment target.

02 / Build Architecture & Pre-Flight

The pipeline requires specific tools to be available on the Jenkins agent before execution. Verifying these dependencies upfront prevents silent failures during the Docker build or push stages.

EXECUTION ARCHITECTURE



PRE-FLIGHT VERIFICATION

```
$ docker --version
Docker version 24.0.2, build cb74dfc
$ groups jenkins
jenkins : jenkins docker
$ git --version
git version 2.34.1
```

GOTCHA

Jenkins must have permission to execute Docker commands. The `jenkins` user must be added to the `docker` group, or the Docker Pipeline plugin must be configured to use a sidecar container. Otherwise, the build stage will fail with a "permission denied" error.

03 / Registry Authentication

To push an image to a private or public Docker Hub repository, Jenkins must authenticate. Credentials must be stored securely in Jenkins and injected at runtime, never hardcoded into the Jenkinsfile.

CREDENTIAL CONFIGURATION

In Jenkins, navigate to **Manage Jenkins** → **Credentials** and add a "Username with password" credential. Use your Docker Hub username and an Access Token (not your account password).

PIPELINE SNIPPET

```
withCredentials([
  usernamePassword(
    credentialsId: 'dockerhub-creds',
    usernameVariable: 'DOCKER_USER',
    passwordVariable: 'DOCKER_PASS'
  )
]) {
  sh 'echo $DOCKER_PASS | docker login -u $DOCKER_USER --
  password-stdin'
}
```

EXECUTION RESULT

```
[Pipeline] stage
[Pipeline] withCredentials
Masking supported pattern matches of $DOCKER_USER or
$DOCKER_PASS
[Pipeline] sh
+ echo ****
Login Succeeded
```

GOTCHA

Docker Hub has deprecated password authentication for CLI push operations. You must generate a Personal Access Token (PAT) from Docker Hub's Account Settings → Security and use that token as the password in Jenkins.

INTERVIEW CHECK

Q: Why use `--password-stdin` instead of `-p <password>` ?

A: Using `-p` on the command line exposes the password in the process list and shell history, which other users on the system could read. `--password-stdin` pipes the token securely via standard input, preventing it from being logged.

04 / Jenkinsfile Construction

The declarative pipeline orchestrates the build, tag, and push sequence. Environment variables capture the image name and build tag, ensuring traceability between the Git commit, the Jenkins build number, and the Docker image.

```
pipeline {
  agent any
  environment {
    IMAGE_NAME = "myusername/app"
    IMAGE_TAG = "build-${env.BUILD_NUMBER}"
  }
  stages {
    stage('Code Checkout') { steps { checkout scm } }
    stage('Docker Build') {
      steps { sh "docker build -t ${IMAGE_NAME}:local ." }
    }
    stage('Image Tagging') {
      steps { sh "docker tag ${IMAGE_NAME}:local ${IMAGE_NAME}:${IMAGE_TAG}" }
    }
    stage('Image Push') {
      steps { sh "docker push ${IMAGE_NAME}:${IMAGE_TAG}" }
    }
  }
}
```

GENERATED ARTIFACT

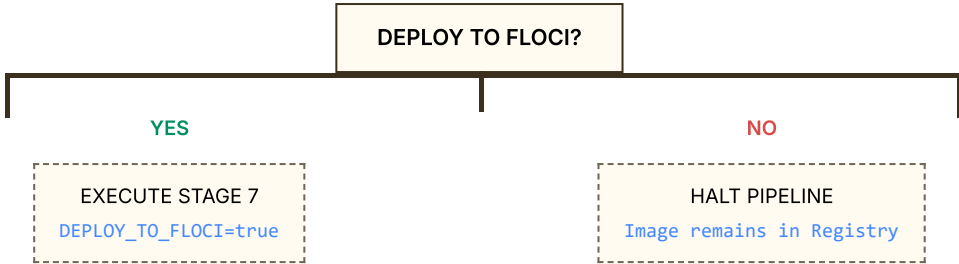
myusername/app:build-123

WHY IT MATTERS

Tagging the image with `build-${BUILD_NUMBER}` creates an immutable, traceable link to the Jenkins execution. If a deployment fails, you can immediately pull the exact image that was deployed and inspect it. Using `:latest` destroys this traceability.

05 / The Deployment Track

Once the image is in the registry, the pipeline can optionally deploy it to a target environment. In this lab, the target is FLOCI, a simulated AWS infrastructure environment. The deployment stage must be conditional to allow for build-only runs.



ENVIRONMENT LANES

BUILD

Docker Build & Push to Registry PASSED

FLOCI

Run application container in simulated AWS RUNNING

CONDITIONAL DEPLOYMENT STAGE

```
stage('Application Deployment') {
  when { expression { return params.DEPLOY_TO_FLOCI == true } }
  steps {
    echo "Deploying ${IMAGE_NAME}:${IMAGE_TAG} to FLOCI ..."
    // FLOCI specific run command here
  }
}
```

06 / Lab: End-to-End Delivery

Execute the pipeline and verify the artifact progression from source code to registry deployment.

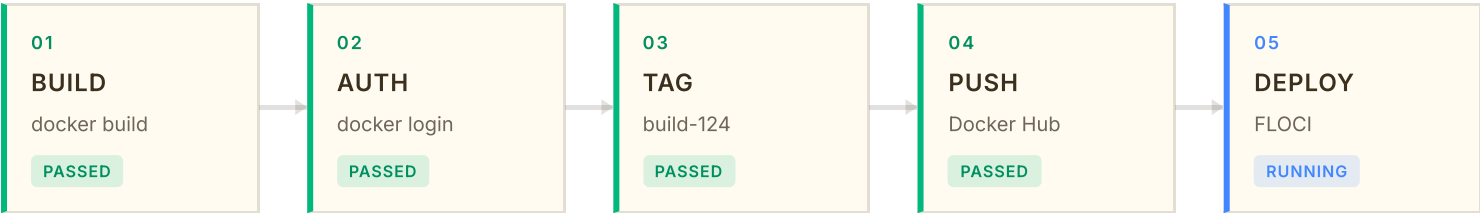
- OBSERVE

The repository contains the Dockerfile and Jenkinsfile. The webhook is configured. We need to verify the image is built and pushed successfully.
- EXECUTE

Trigger the pipeline via a Git push or manual "Build Now". Observe the Jenkins console output for each stage.
- VERIFY

Check Docker Hub for the new tag `build-<N>`. If FLOCI deployment is active, verify the container is running in the simulation.
- PROOF

The application is now packaged as an immutable artifact and delivered to the registry, ready for deployment.



COMMAND

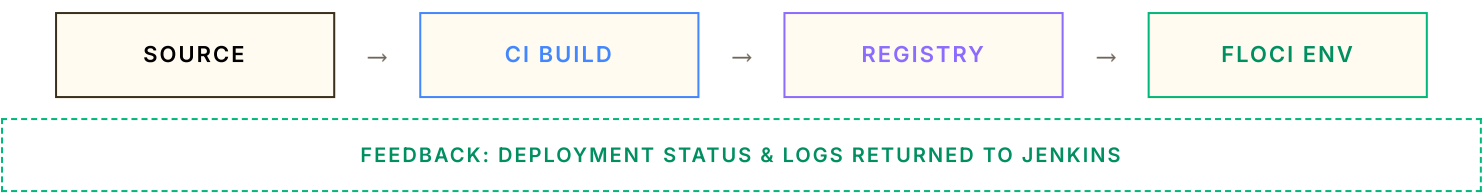
```
$ docker push myusername/app:build-124
```

RESULT

```
The push refers to repository [docker.io/myusername/app]
2c3f4e5d6a7b: Pushed
build-124: digest: sha:4f5b... size: 528
H 02 | 07 / 08
```

07 / System Map & Troubleshooting

The container delivery pipeline is a critical path. Failures in building or pushing artifacts will block all downstream deployments. Use this map to contextualize the Docker build process and resolve common failure states.



TROUBLESHOOTING FLOW

Build fails (permission)	Jenkins user not in docker group	<code>sudo usermod -aG docker jenkins</code>	
SYMPTOM	ROOT CAUSE	RECOVERY	
Deploy fails (not found)	Image tag mismatch	Ensure FLOCI pulls <code>build-\${BUILD_NUMBER}</code>	
Build slow/bloated	Invalid Dockerfile layer cache	Optimize Dockerfile (copy deps first)	

INTERVIEW CHECK

Q: Why tag with the build number instead of the Git commit hash?

A: While commit hashes are immutable, they are not human-readable or sequential. The build number provides a simple, incrementing identifier that directly maps to the Jenkins execution, making it easier to discuss and trace ("build 124 failed").

Up next in Hands-on Labs →

Lab 03: Multibranch Foundations

RELATED BOOKS

Lab 03: Multibranch Foundations
45 min · INT
→ Next in path

Lab 01: Automated Integration Foundations
45 min · INT
→ Previous in path

Jenkins Labs
180 min · ADV
→ Shared tags

Lab 04: Reusable Pipeline Logic
55 min · ADV
→ Shared tags